# Com S // Cpr E // Math 5250

**Numerical Analysis of High-Performance Computing**

**Instructor: Songting Luo**

**Lecture 18: Final Project Information and More OpenMP**

# Outline

1. Final Project Information
2. More OpenMP

# Final Project Information

# Final Project Information

Final project counts **50%** of your grade.

**Final project consists of 4 parts:**

- Project proposal
- Computer code
- Presentation given in-class at the end of the semester ($\sim$ 20 minutes)
- A short writeup

# Final Project Information

Final project counts **50%** of your grade.

**Final project consists of 4 parts:**

- Project proposal
- Computer code
- Presentation given in-class at the end of the semester ($\sim$ 20 minutes)
- A short writeup

**The computer code:**

- Must be written by **you**.
- Will hopefully be relevant to your research and/or research interests
- Can be written in Python, C, C++, or Fortran
- Must be parallelized in some fashion (either OpemMP or MPI)
- Final version of code must be run on the HPC-Class cluster (of course initial writing and debugging can be done on any machine)
- Must show some sort of speed-up by increasing the number of CPUs/cores

# Final Project Information

Final project counts **50%** of your grade.

**Final project consists of 4 parts:**

- Project proposal
- Computer code
- Presentation given in-class at the end of the semester ($\sim$ 20 minutes)
- A short writeup

# Final Project Information

Final project counts **50%** of your grade.

**Final project consists of 4 parts:**

- Project proposal
- Computer code
- Presentation given in-class at the end of the semester ($\sim$ 20 minutes)
- A short writeup

**The writeup:**

- No more than 5 pages with 11-point font and standard margins
- Includes a short project abstract (aka quick summary)
- Includes a short description of your code
- Includes a results section that shows what your code produces and how the code performs with different number of processors/cores

# Final Project Information/Proposal

**Due on April** 19th

**Final Project Proposal:**

- Counts as **10 points** on your final project score
- No more than 2-pages long
- Explain what problem your code is attempting to solve
- State what language(s) you will use in the implementation
- Explain what parts of the code will be parallelized (give ample explanation in this part)
- Will you attempt shared memory (OpenMP) or distributed memory (MPI) parallelization? Explain this with ample detail.

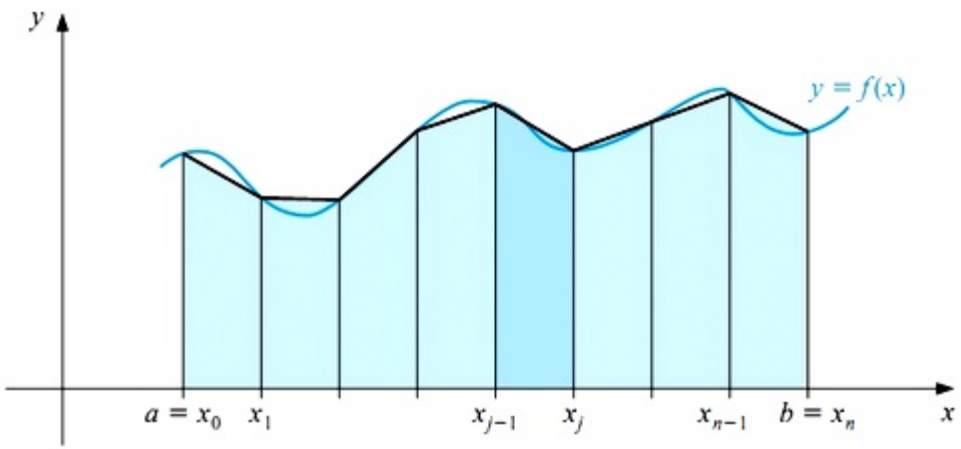
**KEY:** Pick a project that will be beneficial to your work; pick something that is interesting to you.

# How will the final project be graded

- **0–10 points**: Clearly stated and explained project proposal
- **0–10 points**: Does the code produce results in serial?
- **0–20 points**: OpenMP implementation with some speed-up?
- **0–40 points**: MPI implementation with some speed-up?
- **0–20 points**: Presentation clarity (problem statement, how to parallelize, show code snippet(s), show results in parallel)
- **0–20 points**: Final write-up clarity (problem statement, how to parallelize, show code snippet(s), show results in parallel)

# More OpenMP

# Composite Trapezoidal Rule




Consider the integral:

$$I(a,b) = \int_a^b f(x)\,dx = \sum_{i=1}^{N} \int_{x_{i-1}}^{x_i} f(x)\,dx = \sum_{i=1}^{N} I(x_{i-1}, x_i)$$

Let $x_i = a + ih$ where $h = \frac{b-a}{N}$ is the grid spacing

On each sub-interval $[x_{i-1}, x_i]$ we make the following approximation:

$$I(x_{i-1}, x_i) = \int_{x_{i-1}}^{x_i} f(x)\,dx \approx \int_{x_{i-1}}^{x_i} \left[ f(x_{i-1}) \left( \frac{x_i - x}{h} \right) + f(x_i) \left( \frac{x - x_{i-1}}{h} \right) \right] dx$$

# Composite Trapezoidal Rule

$$I(a,b) = \int_a^b f(x)\,dx = \sum_{i=1}^{N} \int_{x_{i-1}}^{x_i} f(x)\,dx = \sum_{i=1}^{N} I(x_{i-1}, x_i)$$

$$I(x_{i-1}, x_i) = \int_{x_{i-1}}^{x_i} f(x)\,dx \approx \int_{x_{i-1}}^{x_i} \left[ f(x_{i-1}) \left( \frac{x_i - x}{h} \right) + f(x_i) \left( \frac{x - x_{i-1}}{h} \right) \right] dx$$

Let $s = \left( \frac{x_i - x}{h} \right) \implies ds = -\frac{1}{h} dx, \qquad x = x_i \implies s = 0, \qquad x = x_{i-1} \implies s = 1$

$$f(x_{i-1}) \int_{x_{i-1}}^{x_i} \left( \frac{x_i - x}{h} \right) dx = h f(x_{i-1}) \int_0^1 s\,ds = \frac{h}{2} f(x_{i-1})$$

Let $s = \left( \frac{x - x_{i-1}}{h} \right) \implies ds = \frac{1}{h} dx, \qquad x = x_i \implies s = 1, \qquad x = x_{i-1} \implies s = 0$

$$f(x_i) \int_{x_{i-1}}^{x_i} \left( \frac{x - x_{i-1}}{h} \right) dx = h f(x_i) \int_0^1 s\,ds = \frac{h}{2} f(x_i)$$

# Composite Trapezoidal Rule

**Composite Trapezoidal Rule:**

$$I(a,b) = \int_a^b f(x)\,dx = \sum_{i=1}^{N} \int_{x_{i-1}}^{x_i} f(x)\,dx = \sum_{i=1}^{N} I(x_{i-1}, x_i)$$

$$I(a,b) \approx T_N := \frac{h}{2} \sum_{i=1}^{N} \left[ f(x_{i-1}) + f(x_i) \right]$$

$$I(a,b) \approx T_N := h \left[ \frac{f(x_0) + f(x_N)}{2} + \sum_{i=1}^{N-1} f(x_i) \right]$$

# Composite Trapezoidal Rule

**Composite Trapezoidal Rule:**

$$I(a,b) = \int_a^b f(x)\,dx = \sum_{i=1}^{N} \int_{x_{i-1}}^{x_i} f(x)\,dx = \sum_{i=1}^{N} I(x_{i-1}, x_i)$$

$$I(a,b) \approx T_N := \frac{h}{2} \sum_{i=1}^{N} \left[ f(x_{i-1}) + f(x_i) \right]$$

$$I(a,b) \approx T_N := h \left[ \frac{f(x_0) + f(x_N)}{2} + \sum_{i=1}^{N-1} f(x_i) \right]$$

**Error Term:**

$$\left| I(a,b) - T_N \right| = \frac{(b-a)}{12} h^2 |f''(\mu_N)| \quad \text{where} \quad a < \mu_N < b$$

# Composite Trapezoidal Rule

**Composite Trapezoidal Rule:**

$$I(a,b) = \int_a^b f(x)\,dx = \sum_{i=1}^{N} \int_{x_{i-1}}^{x_i} f(x)\,dx = \sum_{i=1}^{N} I(x_{i-1}, x_i)$$

$$I(a,b) \approx T_N := \frac{h}{2} \sum_{i=1}^{N} \left[ f(x_{i-1}) + f(x_i) \right]$$

$$I(a,b) \approx T_N := h \left[ \frac{f(x_0) + f(x_N)}{2} + \sum_{i=1}^{N-1} f(x_i) \right]$$

**Error Term:**

$$\left| I(a,b) - T_N \right| = \frac{(b-a)}{12} h^2 |f''(\mu_N)| \quad \text{where} \quad a < \mu_N < b$$

If we double $N$, then $h = (b-a)/N$ is cut in half, and the error is cut by 4.

$$\frac{\left| I(a,b) - T_N \right|}{\left| I(a,b) - T_{2N} \right|} = \frac{h^2 |f''(\mu_N)|}{(h/2)^2 |f''(\mu_{2N})|} = 4 \frac{|f''(\mu_N)|}{|f''(\mu_{2N})|} \approx 4$$

# Composite Trapezoidal Rule: Serial Implementation

$ISUHPC/lectures/lecture18/codes/trap1/main.c:

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int main(int argc, char* argv[])
{
   void usage(const char* prog_name);
   double CompTrap(const double a,const double b,const int N);

   if (argc != 2) { usage(argv[0]); }
   const int N = strtol(argv[1], NULL, 10);
   if (N<1) { usage(argv[0]); }

   const double a = 0.0; const double b = 1.0;
   double T = CompTrap(a,b,N);
   const double Iex = exp(1.0);
   const double err = fabs(T-Iex);

   printf("\n N = %i, T = %23.15e, err = %12.5e\n\n",N,T,err);

   return 0;
}
```

# Composite Trapezoidal Rule: Serial Implementation

$ISUHPC/lectures/lecture18/codes/trap1/main.c:

```
void usage(const char *prog_name)
{
   fprintf(stderr, "usage: %s <num_intervals>\n", prog_name);
   fprintf(stderr, "   num_intervals should be positive\n");
   exit(1);
}

double CompTrap(const double a, const double b, const int N)
{
   double func(const double x);
   double h = (b-a)/((double)N);
   double T = 0.5*(func(a)+func(b));
   double x = a;

   for (int i=1; i<N; i++)
   {
      x += h;
      T += func(x);
   }

   return h*T;
}
```

# Composite Trapezoidal Rule: Serial Implementation

$ISUHPC/lectures/lecture18/codes/trap1/main.c:

```c
double func(const double x)
{
   return (1.0 + exp(x));
}
```

```
$ main.exe 10

 N = 10, T =   2.719713491389315e+00, err =  1.43166e-03

$ main.exe 20

 N = 20, T =   2.718639788925222e+00, err =  3.57960e-04

$ main.exe 40

 N = 40, T =   2.718371321372064e+00, err =  8.94929e-05

$ main.exe 80

 N = 80, T =   2.718304201862090e+00, err =  2.23734e-05
```

# Composite Trapezoidal Rule: Parallel #1

$ISUHPC/lectures/lecture18/codes/trap2/main.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#ifdef _OPENMP
#  include <omp.h>
#endif

int main(int argc, char* argv[])
{
   void usage(const char* prog_name);
   void CompTrap(const double a,const double b,const int N,
                 double* T);

   if (argc != 3) { usage(argv[0]); }
   const int thread_count = strtol(argv[1], NULL, 10);
   const int N = strtol(argv[2], NULL, 10);
   if (thread_count<1 || N<1 || N % thread_count != 0)
   { usage(argv[0]); }

   const double a = 0.0; const double b = 1.0;
   double T = 0.0;
```

# Composite Trapezoidal Rule: Parallel #1

$ISUHPC/lectures/lecture18/codes/trap2/main.c:

```
    const double time1 = omp_get_wtime();
#   pragma omp parallel num_threads(thread_count)
    CompTrap(a,b,N,&T);
    const double time2 = omp_get_wtime();

    double Iex = exp(1.0); double err = fabs(T-Iex);

    printf("\n N = %i, T = %23.15e, err = %12.5e,\n",N,T,err);
    printf(" time = %12.5e\n\n",time2-time1);

    return 0;
}

void usage(const char *prog_name)
{
    fprintf(stderr,"usage: %s <num_threads> <num_intervals>\n",
            prog_name);
    fprintf(stderr,"   num_threads should be positive\n");
    fprintf(stderr,"   num_intervals should be positive\n");
    fprintf(stderr,"   mod(num_intervals,num_threads) != 0\n");
    exit(1);
}
```

# Composite Trapezoidal Rule: Parallel #1

$ISUHPC/lectures/lecture18/codes/trap2/main.c:

```
void CompTrap(const double a, const double b, const int N,
              double* T_global)
{
#   ifdef _OPENMP
    const int my_rank = omp_get_thread_num();
    const int thread_count = omp_get_num_threads();
#   else
    const int my_rank = 0;
    const int int thread_count = 1;
#   endif

    double func(const double x);
    double h = (b-a)/((double)N);

    double local_N = N/thread_count;
    double local_a = a + my_rank*local_N*h;
    double local_b = local_a + local_N*h;

    double T_local = 0.5*(func(local_a)+func(local_b));
    double x_local = local_a;
```

# Composite Trapezoidal Rule: Parallel #1

$ISUHPC/lectures/lecture18/codes/trap2/main.c:

```
    for (int i=1; i<local_N; i++)
    {
        x_local += h;
        T_local += func(x_local);
    }

#   pragma omp critical
    *T_global += h*T_local;
}

double func(const double x)
{
    return (1.0 + exp(x));
}
```

# Composite Trapezoidal Rule: Parallel #1

```
$ main.exe 1 16777216

 N = 16777216, T =   2.718281828458923e+00, err =  1.22125e-13,
 time =  1.54978e-01

$ main.exe 2 16777216

 N = 16777216, T =   2.718281828458913e+00, err =  1.31894e-13,
 time =  8.03759e-02

$ main.exe 4 16777216

 N = 16777216, T =   2.718281828458936e+00, err =  1.09246e-13,
 time =  4.53699e-02

$ main.exe 8 16777216

 N = 16777216, T =   2.718281828459044e+00, err =  8.88178e-16,
 time =  3.75550e-02
```

# Composite Trapezoidal Rule: Parallel #2

$ISUHPC/lectures/lecture18/codes/trap3/main.c:

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#ifdef _OPENMP
#  include <omp.h>
#endif

int main(int argc, char* argv[])
{
   void usage(const char* prog_name);
   double LocalCompTrap(const double a,const double b,const
       int N);

   if (argc != 3) { usage(argv[0]); }
   const int thread_count = strtol(argv[1], NULL, 10);
   const int N = strtol(argv[2], NULL, 10);
   if (thread_count<1 || N<1 || N % thread_count != 0)
   { usage(argv[0]); }

   const double a = 0.0; const double b = 1.0;
   double T = 0.0;
```

# Composite Trapezoidal Rule: Parallel #2

$ISUHPC/lectures/lecture18/codes/trap3/main.c:

```
    const double time1 = omp_get_wtime();
#   pragma omp parallel num_threads(thread_count)
    {
       double T_local = LocalCompTrap(a,b,N);
#      pragma omp critical
       T += T_local;
    }
    const double time2 = omp_get_wtime();

    double Iex = exp(1.0); double err = fabs(T-Iex);

    printf("\n N = %i, T = %23.15e, err = %12.5e,\n",N,T,err);
    printf(" time = %12.5e\n\n",time2-time1);

    return 0;
}
```

# Composite Trapezoidal Rule: Parallel #2

$ISUHPC/lectures/lecture18/codes/trap3/main.c:

```
double LocalCompTrap(const double a, const double b,
                     const int N)
{
#   ifdef _OPENMP
    const int my_rank = omp_get_thread_num();
    const int thread_count = omp_get_num_threads();
#   else
    const int my_rank = 0;
    const int int thread_count = 1;
#   endif

    double func(const double x);
    double h = (b-a)/((double)N);

    double local_N = N/thread_count;
    double local_a = a + my_rank*local_N*h;
    double local_b = local_a + local_N*h;

    double T_local = 0.5*(func(local_a)+func(local_b));
    double x_local = local_a;
```

# Composite Trapezoidal Rule: Parallel #2

$ISUHPC/lectures/lecture18/codes/trap3/main.c:

```
    for (int i=1; i<local_N; i++)
    {
        x_local += h;
        T_local += func(x_local);
    }

    return h*T_local;
}

double func(const double x)
{
    return (1.0 + exp(x));
}
```

# Composite Trapezoidal Rule: Parallel #2

```
$ main.exe 1 16777216

 N = 16777216, T =   2.718281828458923e+00, err =  1.22125e-13,
 time =  1.56998e-01

$ main.exe 2 16777216

 N = 16777216, T =   2.718281828458913e+00, err =  1.31894e-13,
 time =  8.08611e-02

$ main.exe 4 16777216

 N = 16777216, T =   2.718281828458936e+00, err =  1.09246e-13,
 time =  4.32181e-02

$ main.exe 8 16777216

 N = 16777216, T =   2.718281828459044e+00, err =  8.88178e-16,
 time =  3.80840e-02
```

# Composite Trapezoidal Rule: Parallel #3

$ISUHPC/lectures/lecture18/codes/trap3/main.c:

```
    double T = 0.0;

    const double time1 = omp_get_wtime();
#   pragma omp parallel num_threads(thread_count)
    {
        double T_local = LocalCompTrap(a,b,N);
#       pragma omp critical
        T += T_local;
    }
    const double time2 = omp_get_wtime();
```

$ISUHPC/lectures/lecture18/codes/trap4/main.c:

```
    double T = 0.0;

    const double time1 = omp_get_wtime();

#   pragma omp parallel num_threads(thread_count) \
    reduction(+: T)
    T += LocalCompTrap(a,b,N);

    const double time2 = omp_get_wtime();
```

# Composite Trapezoidal Rule: Parallel #4

$ISUHPC/lectures/lecture18/codes/trap5/main.c:

```
int main(int argc, char* argv[])
{
   void usage(const char* prog_name);
   double CompTrap(const double a,const double b,
                   const int N,const int thread_count);

   if (argc != 3) { usage(argv[0]); }
   const int thread_count = strtol(argv[1], NULL, 10);
   const int N = strtol(argv[2], NULL, 10);
   if (thread_count<1 || N<1 || N % thread_count != 0)
   { usage(argv[0]); }

   const double a = 0.0; const double b = 1.0;
   const double time1 = omp_get_wtime();
   double T = CompTrap(a,b,N,thread_count);
   const double time2 = omp_get_wtime();
   double Iex = exp(1.0); double err = fabs(T-Iex);

   printf("\n N = %i, T = %23.15e, err = %12.5e,\n",N,T,err);
   printf(" time = %12.5e\n\n",time2-time1);

   return 0;
}
```

# Composite Trapezoidal Rule: Parallel #4

$ISUHPC/lectures/lecture18/codes/trap5/main.c:

```
double CompTrap(const double a, const double b,
                const int N, const int thread_count)
{
   double func(const double x);
   double h = (b-a)/((double)N);

   double T = 0.5*(func(a)+func(b));
#  pragma omp parallel for num_threads(thread_count)    \
   reduction(+: T)
   for (int i=1; i<N; i++)
   {
      T += func(a+i*h);
   }

   return h*T;
}
```

# Composite Trapezoidal Rule: Parallel #4

```
$ main.exe  1 16777216

 N = 16777216, T =   2.718281828458923e+00, err =  1.21680e-13,
 time =  1.58700e-01

$ main.exe 2 16777216

 N = 16777216, T =   2.718281828458913e+00, err =  1.31894e-13,
 time =  8.26621e-02

$ main.exe 4 16777216

 N = 16777216, T =   2.718281828458935e+00, err =  1.09690e-13,
 time =  4.39019e-02

$ main.exe 8 16777216

 N = 16777216, T =   2.718281828459044e+00, err =  8.88178e-16,
 time =  3.56541e-02
```

# Lab assignment

- Implement Simpsion's rule with OPENMP.
- Application: Compute the price of a European call option, compare it with the Black-Scholes formula.
  - The option price:

$$C = e^{-rT} \int_0^{\infty} \max\{S-K, 0\} p(S) dS \approx e^{-rT} \int_K^{S_{\max}} \{S-K\} p(S) dS,$$

where

  - $S$: asset price at maturity
  - $K$: strike price
  - $r$: risk rate
  - $T$: time to maturity
  - $p(S) = \frac{1}{S\sigma\sqrt{2\pi T}} e^{-\frac{(\log S - \mu)^2}{2\sigma^2 T}}$ : probability density function of $S$; with $\mu = \log(S_0) + (r - \sigma^2/2)T$.
  - truncate at $S_{\max} \approx 5S_0$.

# Lab assignment

- Black-Scholes Exact Formula:

$$C = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2),$$

where

$$d_1 = \frac{\log(S_0/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}}$$

$$d_2 = d_1 - \sigma\sqrt{T}$$

$$\Phi(x) = \frac{1}{2}\left(1 + \mathrm{erf}\left(\frac{x}{\sqrt{2}}\right)\right)$$

- Test case: $S_0 = 100, K = 100, r = 0.05, T = 1, \sigma = 0.2$.
- Use $N = 1e7$. And Theads $= 1, 2, 4, 8, 16$. Submit source codes and screenshots.